



life.augmented

Add encryption service of sensitive data

Tomas Team

6

Add encryption service of sensitive data

- Timing : 45 minutes
- Purpose
 - Show Secure storage provisioning and key usage in crypto
- Process
 - Generate a preformatted key-store containing a secret key
 - Reinstall the secure manager containing the new key store
 - Use psa crypto api using provisioned key to encrypt the data
 - Check encrypted output with a python tool on PC
- Payoff
 - PSA crypto API available thanks to secure manager. Give an idea of how secure and powerful is the secure manager

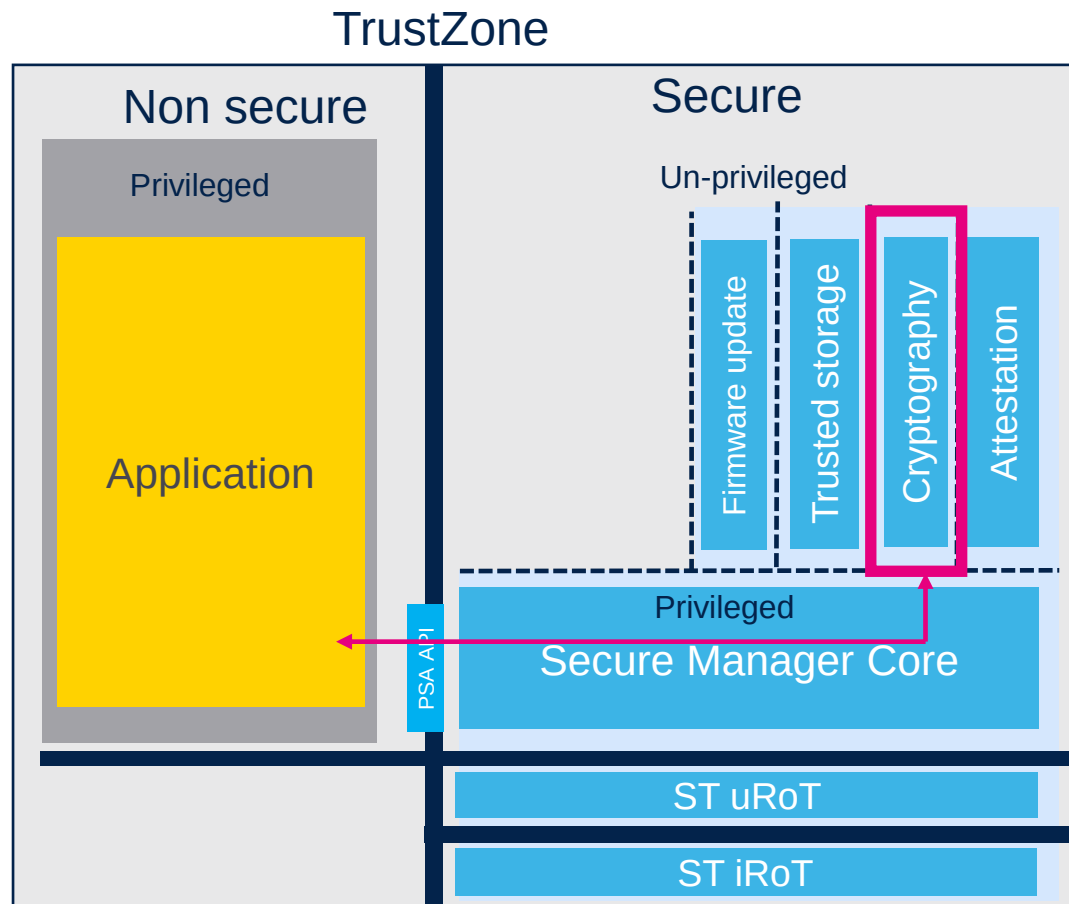


Security improvement

- Problematic : Timestamp information is confidential but our channel of communication is not secure...
- How to address this issue ?
 - We need to add the encryption of the Timestamp information on the target but also the capability to decode it on our PC.
- Cryptographic tool chosen : AES-CBC (Symmetric cryptography)
- Task
 - we need to put encryption key in the target securely and protect it against any direct access
 - we need to add the encryption of the data before transmission via the USART



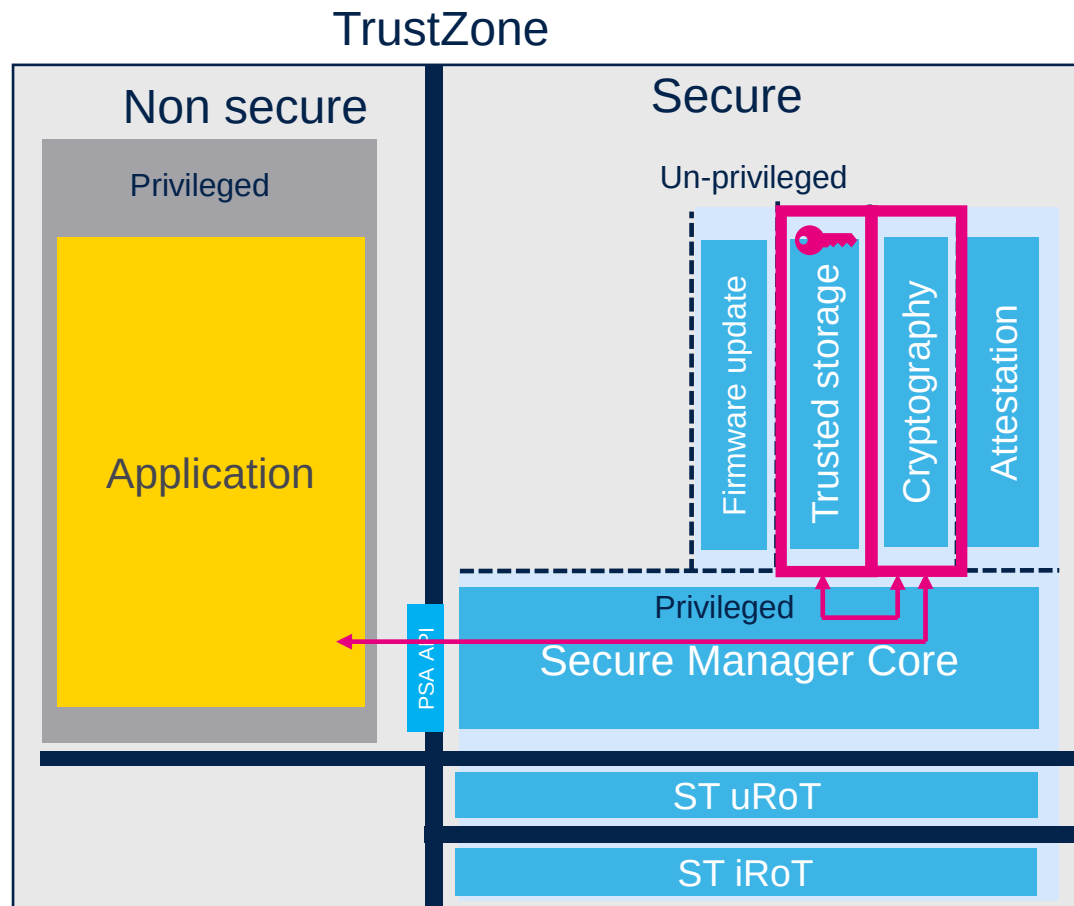
Encryption of the data



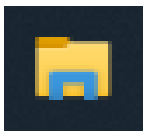
- We will use the cryptography secure services
- PSA API : `psa_cipher_encrypt`
- The AES key 🔑 use for encryption could be stored in the application or could be stored in the internal trusted storage



Store the encryption key provisioned at production

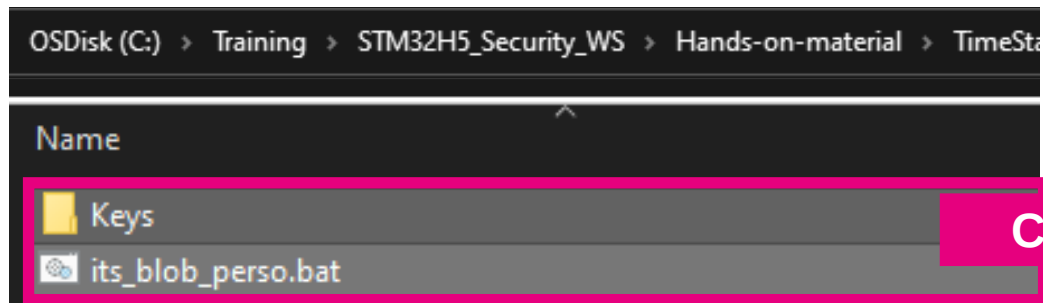


- The internal trusted storage could be provisioned at production level (at the same time that the secure manager)
- The application will call the `psa_cipher_encrypt` with an index to specify which key to use.

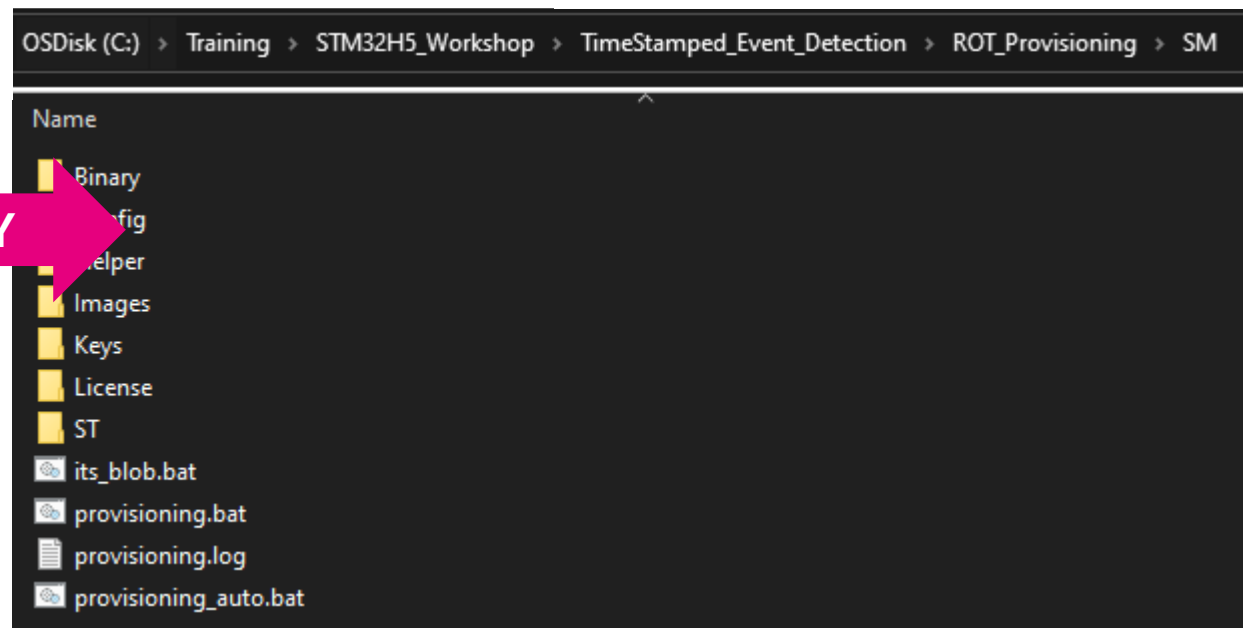


Add the encryption key and its blob script

Material location : C:\Training\STM32H5_Security_WS\Hands-on-material\TimeStamped_Event_Detection\ROT_Provisioning\SM



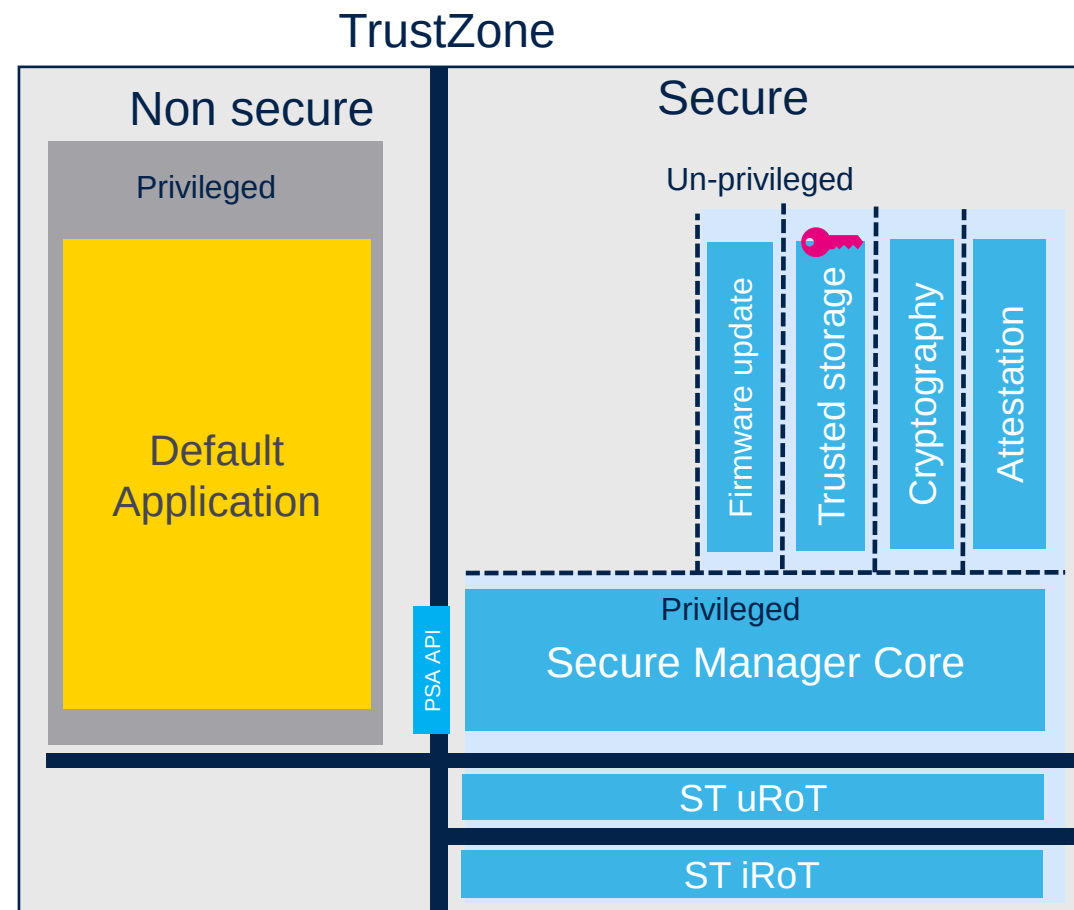
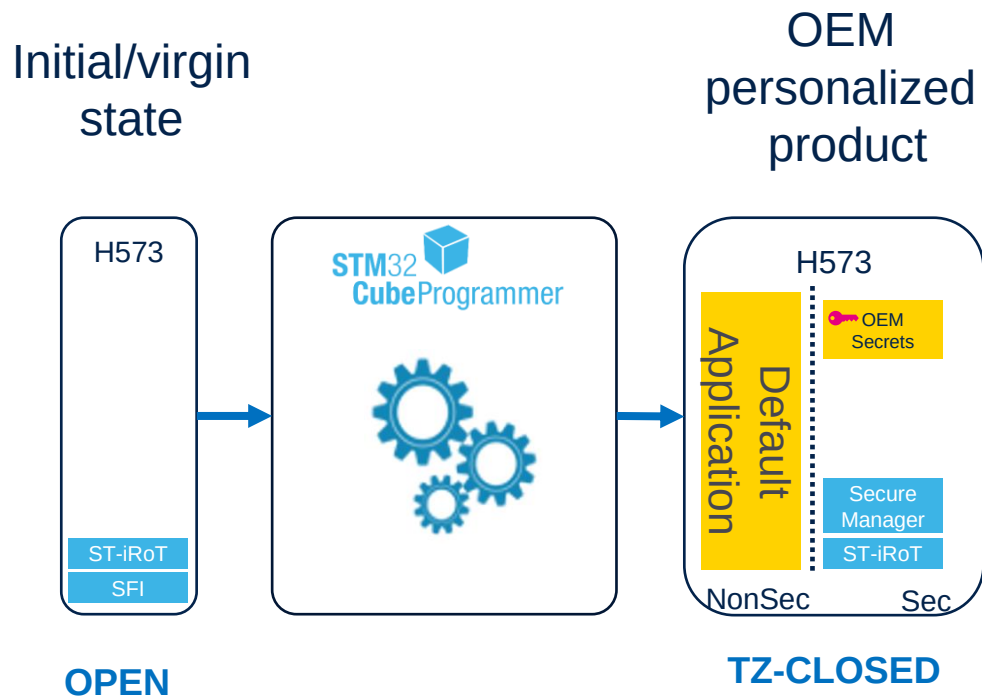
COPY

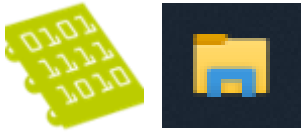


Material destination :
C:\Training\STM32H5_Security_WS\Hands-on\TimeStamped_Event_Detection\ROT_Provisioning\SM



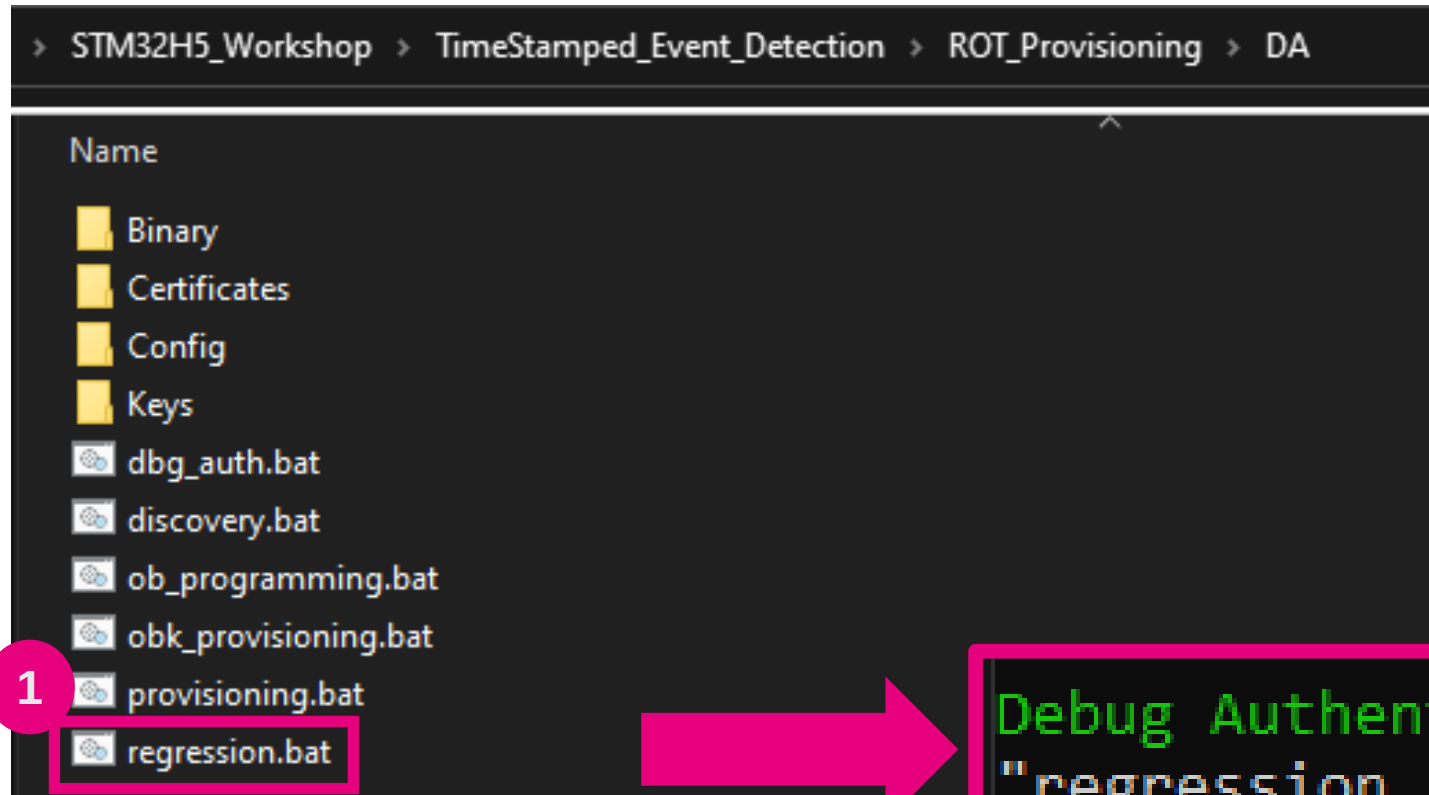
Step1 : provision the AES key in the ITS





First uninstall the current configuration

Please stop the debugger from previous hands-on if needed



Debug Authentication Success
"regression script success"



How to personalize the Factory ITS ?

- Installation script log :

```
* Factory ITS blob preparation:
  Personalize (if/as needed) ITS blob content (in \ROT_Provisioning\its_blob.bat)
  then regenerate the blob (ROT_Provisioning\SM\Binary\ITS_Factory_Blob.bin) by running its_blob script.
  Press any key to continue...
```

- Default script example is

C:\Training\STM32H5_Security_WS\Hands-on\TimeStamped_Event_Detection\ROT_Provisioning\SM\its_blob.bat

- Tools use by this script
STM32Cube_FW_H5_V1.1.1\Utilities\PC_Software\ITSbuilder



its_blob_perso.bat

AES Key value



ROT_Provisioning\SM\Keys\AES_256bits.bin

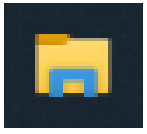
ITS Builder



ROT_Provisioning\SM\Binary\ITS_Factory_Blob.bin

Parameters:

- Type of the key (AES -b 256 -a CBC)
- Index of the key (0x23)
- Capability of the key (ENCRYPT)



Generate the ITS_Factory_Blob.bin

- Execute

Training > STM32H5_Workshop > TimeStamped_Event_Detection > ROT_Provisioning > SM

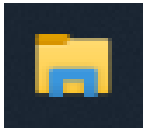
Name

- Binary
- Config
- Helper
- Images
- Keys
- License
- ST
- its_blob.bat
- its_blob.log
- its_blob_perso.bat
- provisioning.bat
- provisioning.log
- provisioning_auto.bat

1

C:\WINDOWS\system32\cmd.exe

```
run ITSbuilder with Windows executable
=====
===== ITS blob generation successful
=====
```



Please check its_blob_perso.log

ITSbuilder v:1.1.2

version: 0x0001

data area: 0

keys area: 1

AES key -256b

Id: 0x00000023

owner: 0xFFFFFFFF

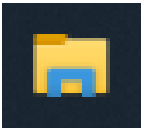
persistence: PERSISTENT, location: 0x000000000000

usage(s): ENCRYPT

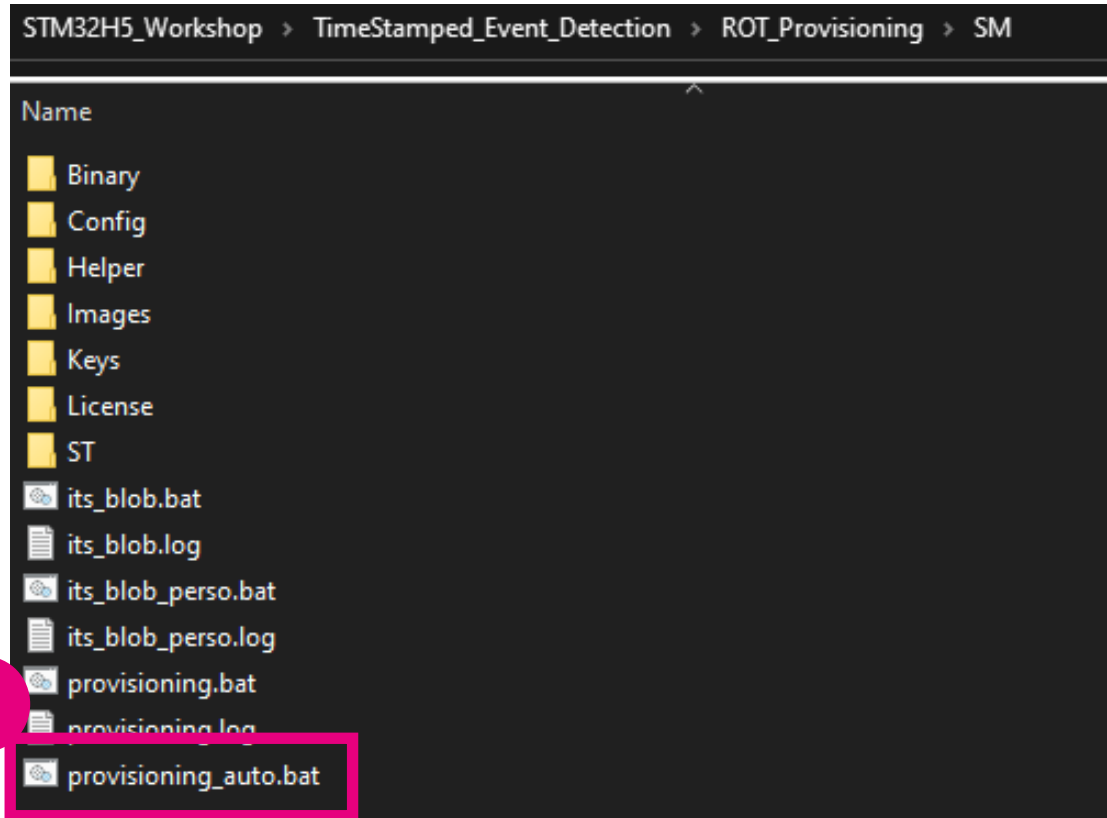
argument(s): CBC (0x4404000)

data length: 32

value: 0x463b412911767d57a0b33969e674ffe7845d313b88c6fe312f3d724be68e1fca



Install the secure manager





```
* Factory ITS blob preparation:
  Personalize (if/as needed) ITS blob content (in \ROT_Provisioning\its_blob.bat)
  then regenerate the blob (ROT_Provisioning\SM\Binary\ITS_Factory_Blob.bin) by running its_blob script.
  Press any key to continue...
```

```
* SFI global license configuration:
  From TrustedPackageCreator (tab H5-License Gen)
  Select SFIG in the "License Type" list
  Open encryption key file (\ROT_Provisioning\SM\Keys\SFI_Encryption_Key.bin)
  and nonce file (\ROT_Provisioning\SM\Keys\SFI_Encryption_Nonce.bin),
  then regenerate them (if/as needed) with same name.
  Select Output license file (\ROT_Provisioning\SM\Keys\SFI_Global_License.bin)
  then Generate License (if/as needed) with same name.
  Press any key to continue...
```

```
* Flash layout processing
  Processing...
```

```
* SFI file generation:
  Processing...
```

```
    Successful SFI file generation
```

Step 2 : Installation

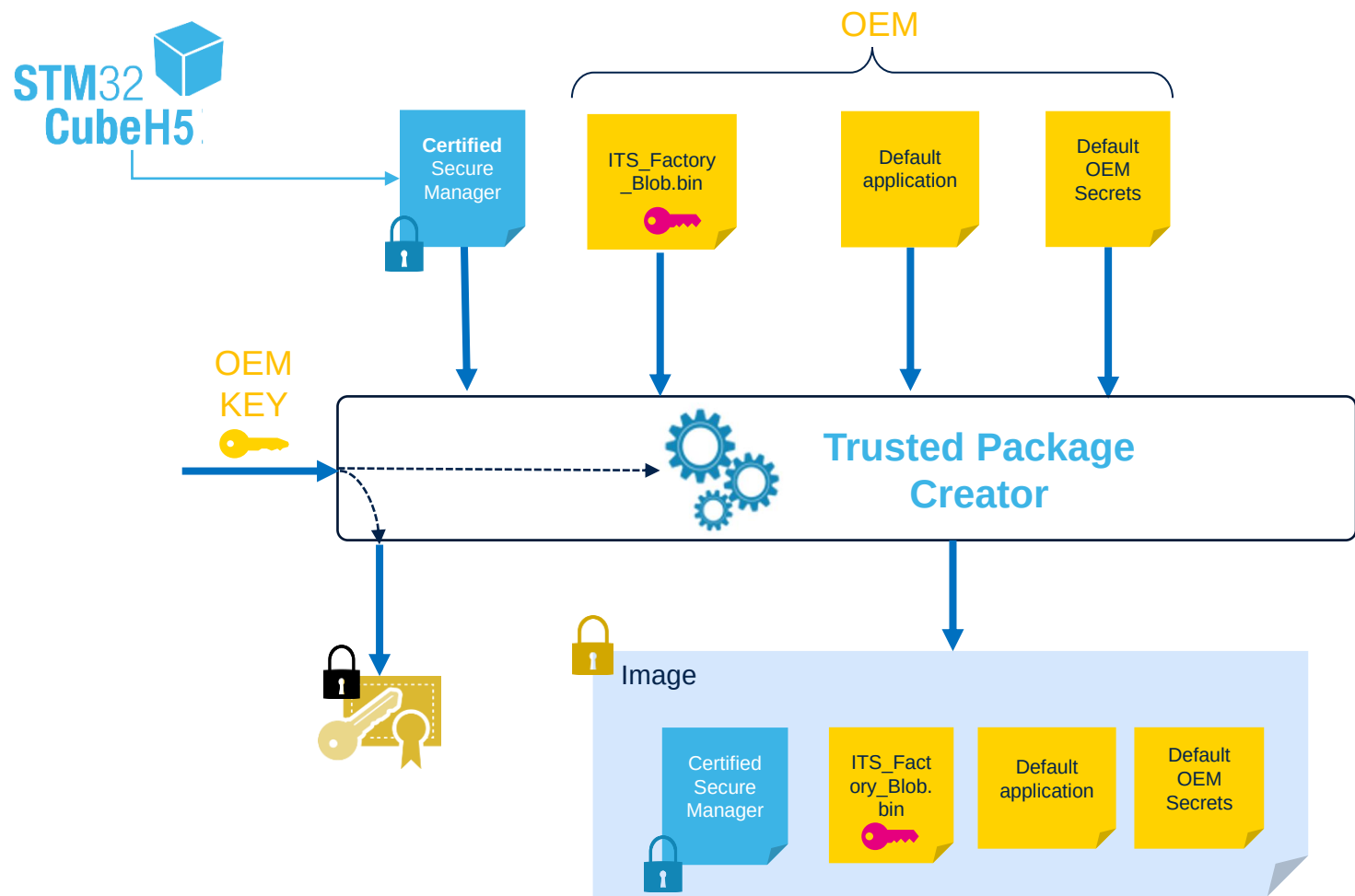
```
* BOOT0 pin should be disconnected from VDD
  (STM32H573I-DK: set SW1 to position 0)
  Press any key to continue...
```

```
* Secure installation
  Installation starting (might take up to 15 seconds)...
  Installation completion...
  Successful installation
```

```
====
==== The board is correctly configured with ST Secure Manager Package
====
```



SFI image generation

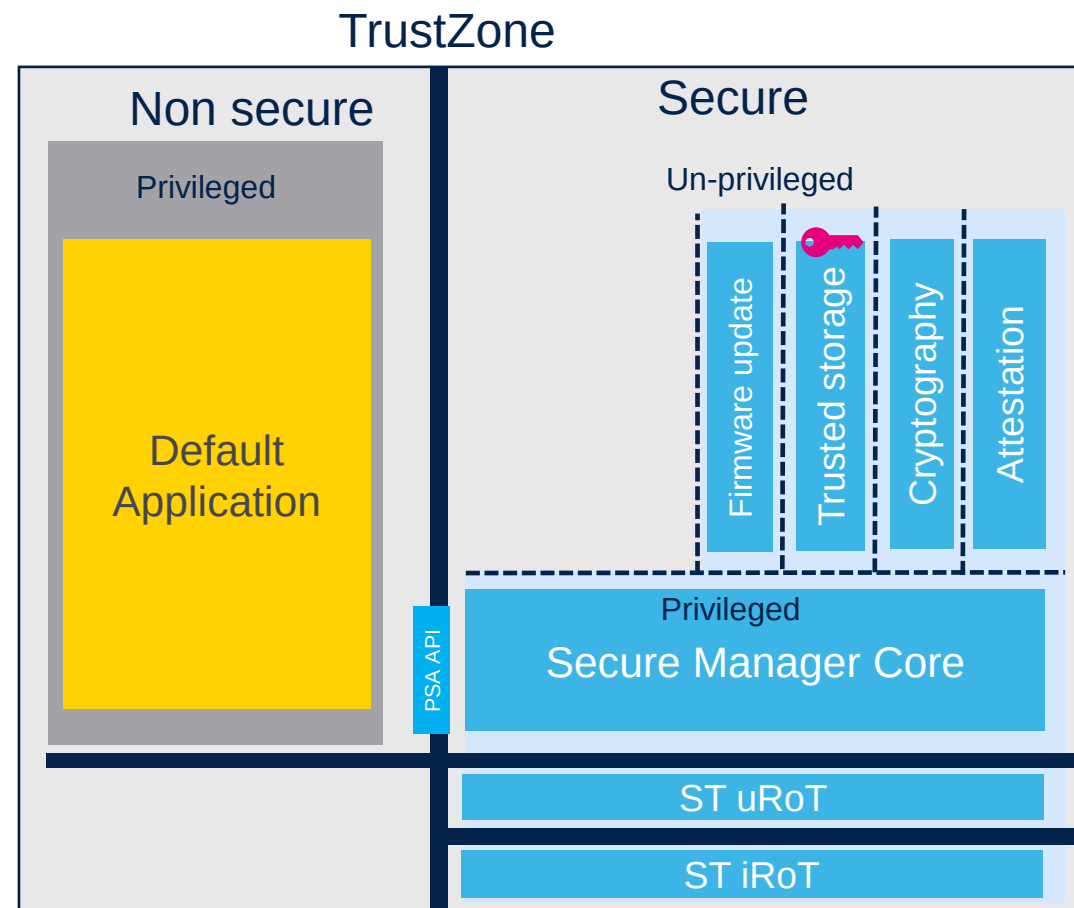
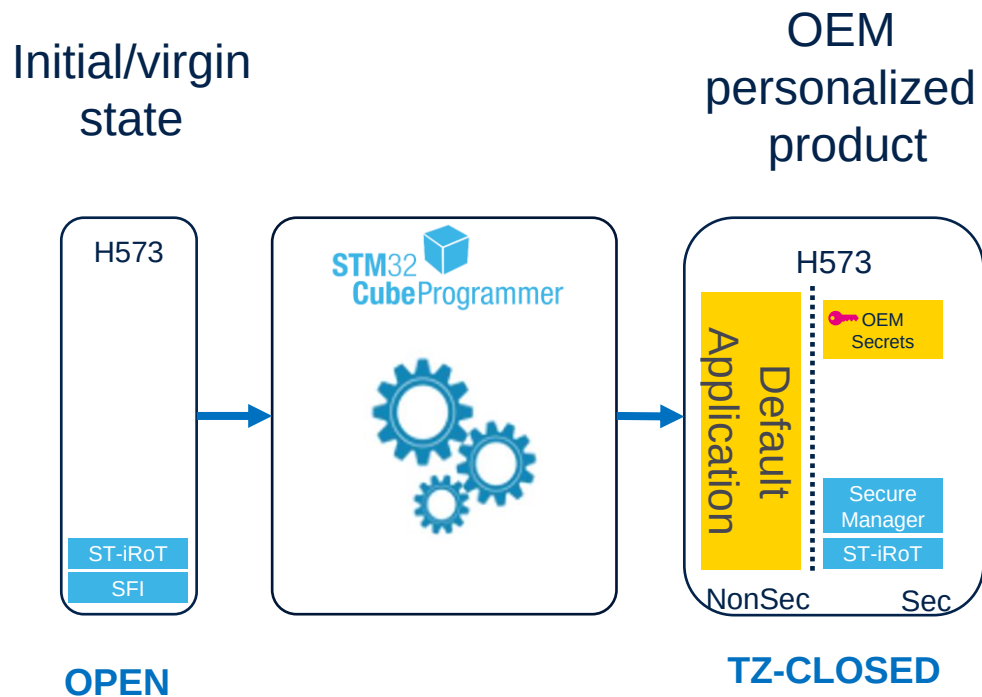


Protected by ST specific Key

Protected by OEM Key



Step1 : provision the AES key in the ITS done !





Step 2 : modify the application to encrypt data

- We need to include the PSA crypto API thanks CubeMX
- Modify our application to call : `psa_cipher_encrypt`

Add PSA Cryptography API

Pinout & Configuration

Clock Configuration

Software Packs

Pinout

SECURE_MANAGER_API Mode and Configuration

Mode

Runtime contexts:

Cortex-M33 secure	Cortex-M33 non secure
	☒

Secure Manager services

☒ Cryptography

☐ Initial Attestation

☐ Internal trusted storage

☐ Firmware

Secure Manager profile

Large

Categories

A->Z

Middleware and Software Packs

	M33S	M33NS
FILEX		☒
FP-SNS-SMARTAG2		
I-CUBE-CANOPEN		
I-CUBE-Cesium		
I-CUBE-wolfSSL		
I-Cube-SoM-uGOAL		
LEVELX		☒
NETXDUO		☒
☒ SECURE_MANAGE...		☒
THREADX		☒

5

GENERATE CODE

4

3

4

MX Code Generation

i

The Code is successfully generated under :

C:/Training/STM32H5_Security_WS/H.../TimeStamped_Event_Detection

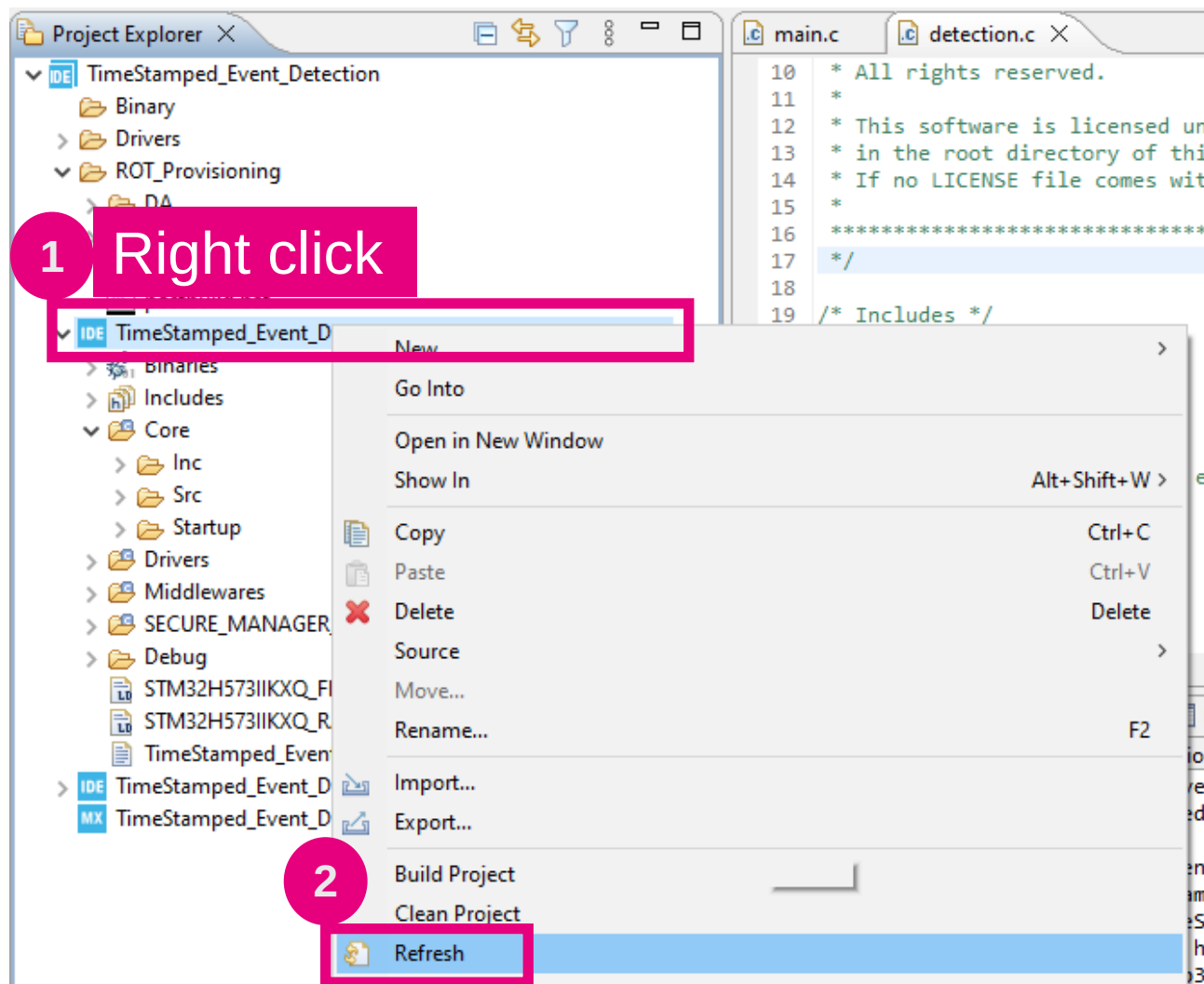
Project language : C

Open Folder

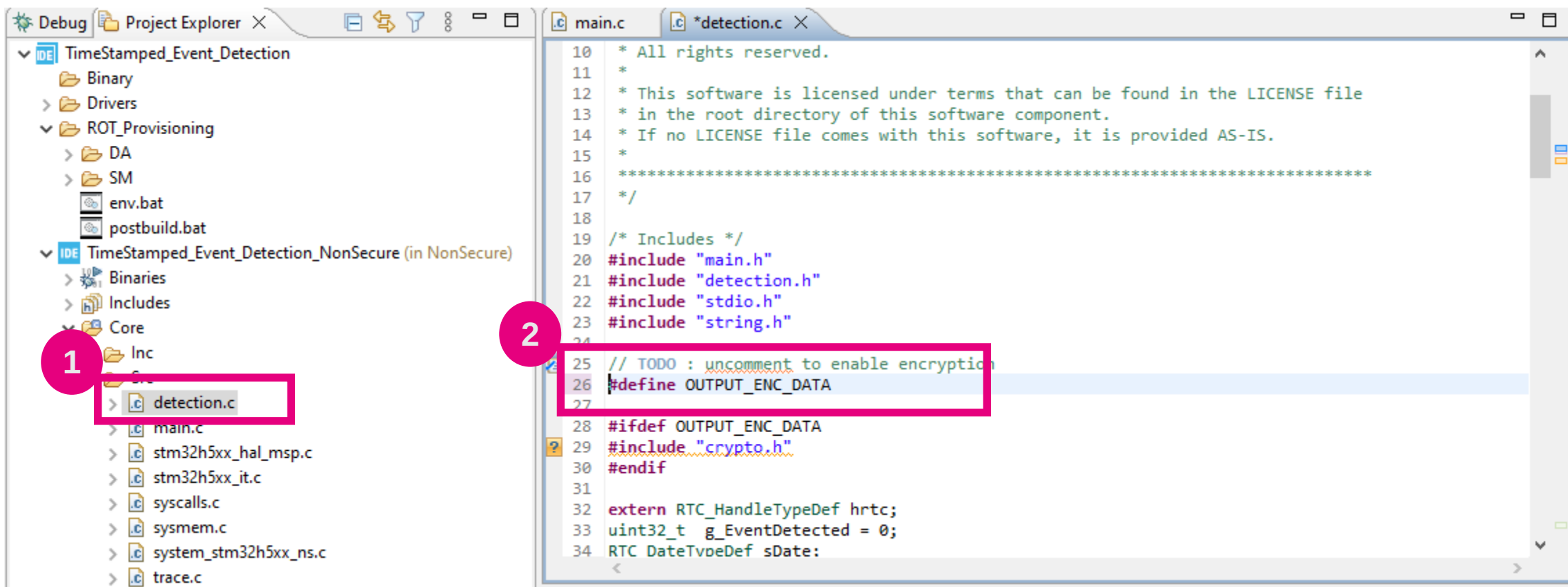
Open Project

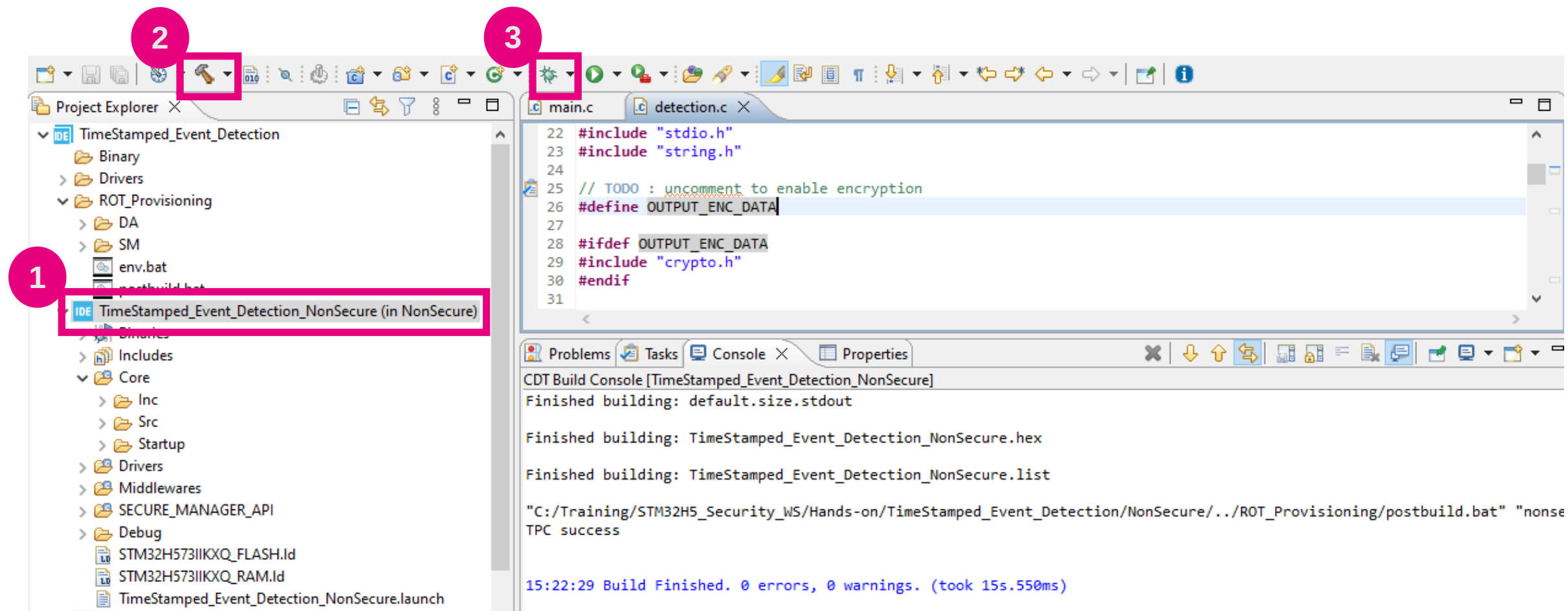
Close

Refresh the project !



Activate the encryption in the code





1. Select the project **TimeStamped_Event_Detection_NonSecure (in NonSecure)** in the Project Explorer.

2. Click the **Build** button (hammer icon) in the toolbar.

3. Click the **Run** button (green play icon) in the toolbar.

```

22 #include "stdio.h"
23 #include "string.h"
24
25 // TODO : uncomment to enable encryption
26 #define OUTPUT_ENC_DATA
27
28 #ifdef OUTPUT_ENC_DATA
29 #include "crypto.h"
30 #endif
31

```

CDT Build Console [TimeStamped_Event_Detection_NonSecure]

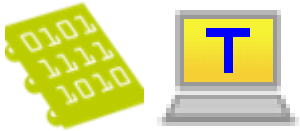
Finished building: default.size.stdout

Finished building: TimeStamped_Event_Detection_NonSecure.hex

Finished building: TimeStamped_Event_Detection_NonSecure.list

"C:/Training/STM32H5_Security_WS/Hands-on/TimeStamped_Event_Detection/NonSecure/../../ROT_Provisioning/postbuild.bat" "nonse
TPC success

15:22:29 Build Finished. 0 errors, 0 warnings. (took 15s.550ms)



Test application....

```
VT COM77 - Tera Term VT
File Edit Setup Control Window Help
*****
Timestamped event detection version 1
*****

*****Event detected (enc) *****
0281057747163d65016fb38d0b7fd0bb7baba780e14b2e75162fda9f196c5f168acf15f50ec1b602e3cf66e91dd88436731af608f54d07bbf11e386
76a0e23e7186c47c95ed47e7142384d30c40019a9

*****Event detected (enc) *****
f16054dbc0d843614e56417623f15d29134021a1d2a3b8d8ba82597a4edd19128656d0e165abc65f4299569a0125d084cee958e06d61e52db0f72b5
531a82735977f74abc544ce8325804100cddaba15

*****Event detected (enc) *****
bfb9db93e06d60da061206c0c700a3137e68846913786b3e0ca1ff8a5435aff8b99873c95c3ca7b651a1b8d423784b1022c5046b73d4165c85840bb
b4486d970f0e3d0095d4c4cb3a0fe1a6729a5b400
□
```

Let's decrypt the data !

- Launch the decrypt utility :

1

< Windows (C:) > Training > STM32H5_Security_WS > Hands-on-material > Tools

2

Name
teraterm-4.106
decrypt.exe
decrypt.py
decrypt_ui.exe
decrypt_ui.py

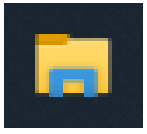
STM32H5 Workshop AES CBC Decrypt

Input (nonce + ciphertext):

Key:

463B412911767D57A0B33969E674FFE7845D313B88C6FE312F3D724BE68E1FCA

Decrypt



Let's decrypt the data !

```
VT COM77 - Tera Term VT
File Edit Setup Control Window Help
*****
Timestamped event detection version 1
*****
*****Event detected <enc> *****
0281057747163d65016fb38d0b7fd0bb7baba780e14b2e75162fda9f196c5f168acf15f50ec1b602e3cf66e91dd88436731af608f54d07bbf11e386
76a0e23e7186c47c95ed47e7142384d30c40019a9
*****Event detected <enc> *****
f16054dbc0d843614e56417623f15d29134021a1d2a3b8d8ba82597a4edd19128656d0e165abc65f4299569a0125d084cee958e06d61e52db0f72b5
531a82735977f74abc544ce8325804100cddaba15
*****Event detected <enc> *****
bfb9db93e06d60da061206c0c700a3137e68846913786b3e0ca1ff8a5435aff8b99873c95c3ca7b651a1b8d423784b1022c5046b73d4165c85840bb
b4486d970f0e3d0095d4c4cb3a0fe1a6729a5b400
```

1

CTL+C

CTL+V

2

STM32H5 Workshop AES CBC Decrypt

Input (nonce + ciphertext):

bfb9db93e06d60da061206c0c700a3137e68846913786b3e0ca1ff8a5435aff8b99873c95c3ca7b651a1b8d423784b1022c5046b73d4165c85840bb
b4486d970f0e3d0095d4c4cb3a0fe1a6729a5b400

Key:

463B412911767D57A0B33969E674FFE7845D313B88C6FE312F3D724BE68E1FCA

Detect button

Time : 00:21:10

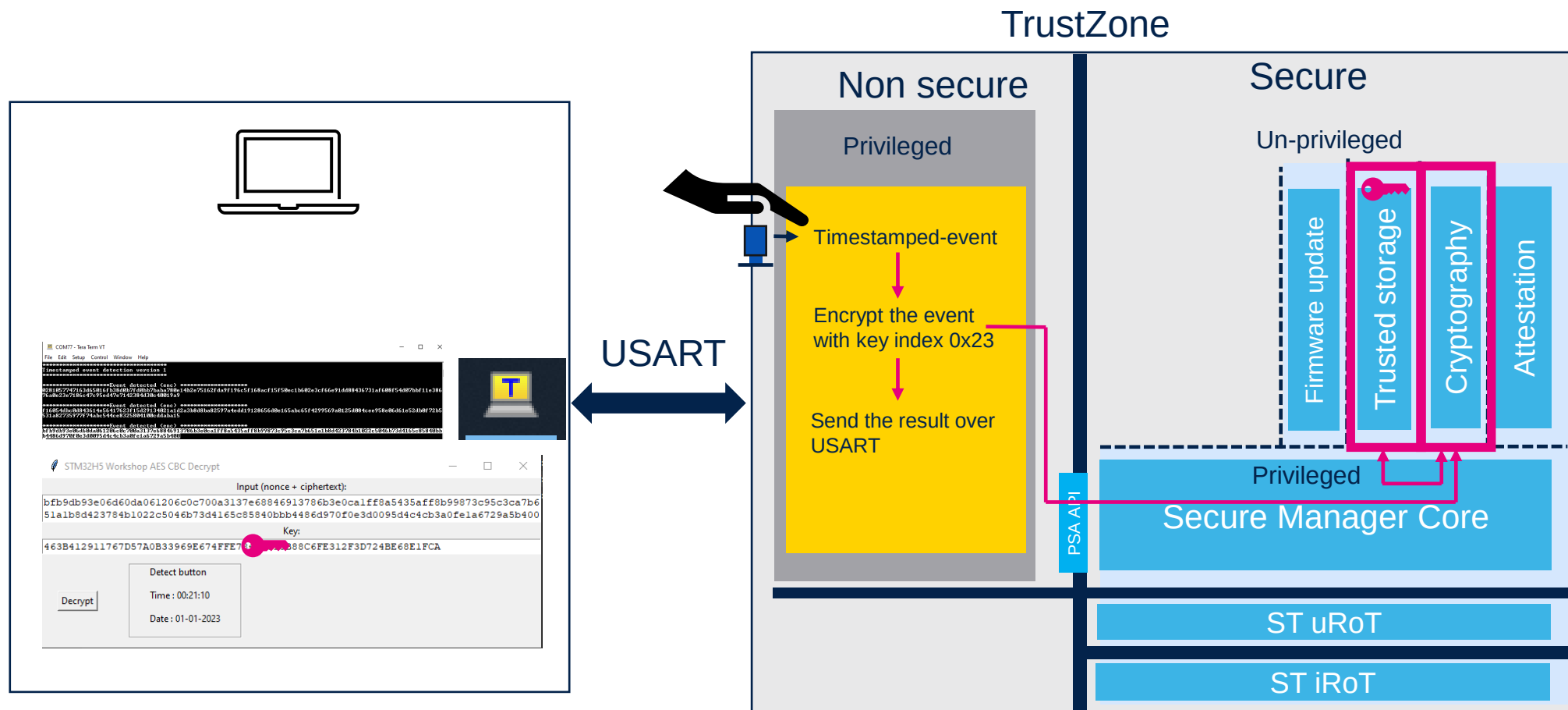
Date : 01-01-2023

Decrypt

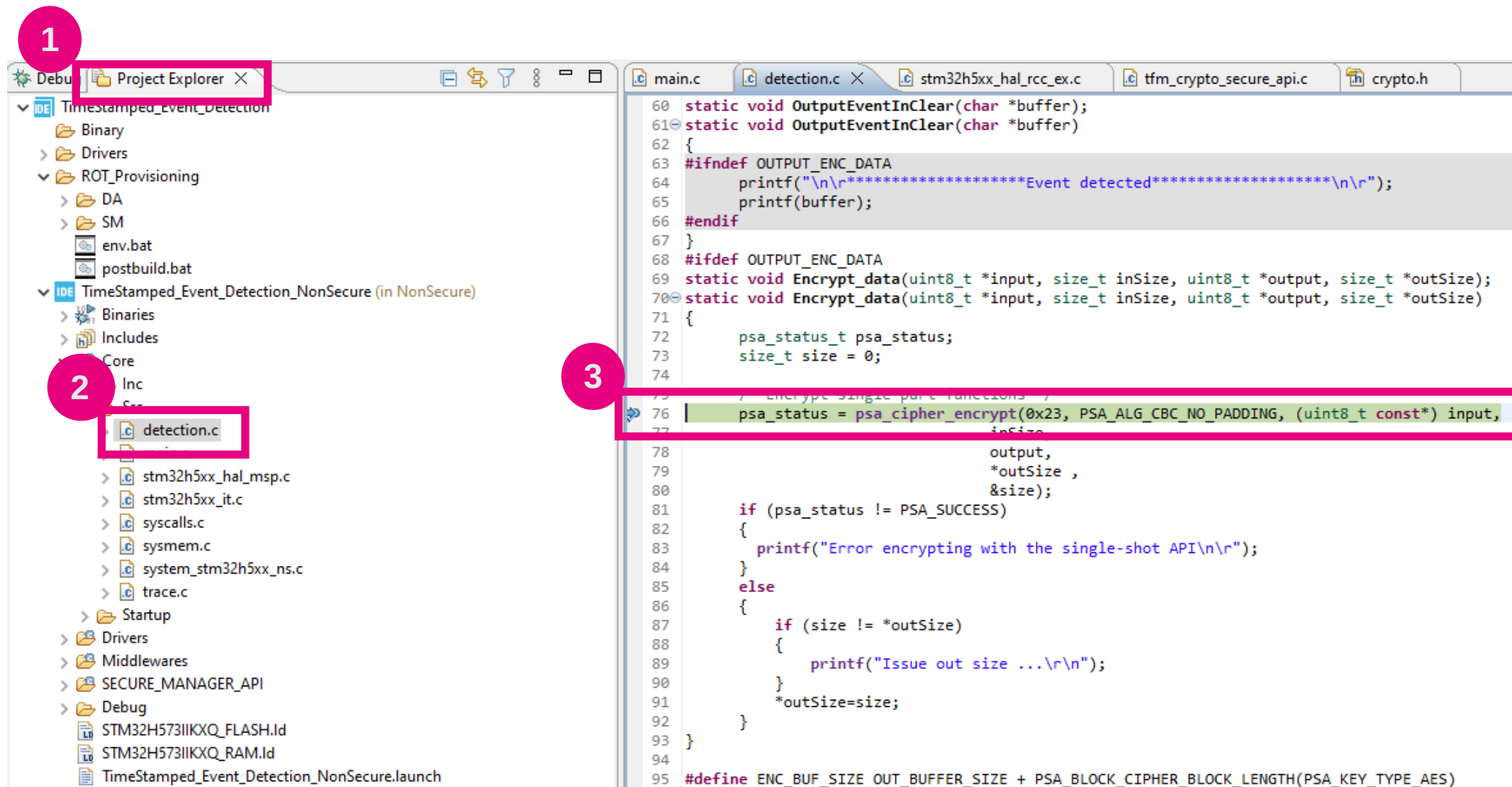
3



Store the encryption key provisioned at production

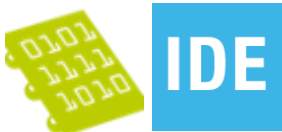


Put a breakpoint line 76 in detection.c



The screenshot shows an IDE interface with three numbered steps to set a breakpoint:

- 1**: The **Project Explorer** tab is selected on the left.
- 2**: The file **detection.c** is selected in the project tree under **TimeStamped_Event_Detection_NonSecure**.
- 3**: A breakpoint is set on line 76 of **detection.c**, which contains the line: `psa_status = psa_cipher_encrypt(0x23, PSA_ALG_CBC_NO_PADDING, (uint8_t const*) input,`



Press the user button of the board

Debug X Project Explorer

IDE TimeStamped_Event_Detection_NonSecure [STM32 C/C++ Application]

TimeStamped_Event_Detection_NonSecure.elf [cores: 1]

Thread #1 [main] 1 [core: 1] (Suspended : Breakpoint)

- Encrypt_data() at detection.c:76 0x806ea6e
- OutputEventInEncrypted() at detection.c:103 0x806eaf2
- ProcessEventDetection() at detection.c:120 0x806eb58
- main() at main.c:112 0x806eba6

arm-none-eabi-gdb (12.1.90.20220802)

ST-LINK (ST-LINK GDB server)

main.c detection.c X stm32h5xx_hal_rcc_ex.c tfm_crypto_secure_api.c crypto.h

```
60 static void OutputEventInClear(char *buffer);
61 static void OutputEventInClear(char *buffer)
62 {
63 #ifndef OUTPUT_ENC_DATA
64     printf("\n\r*****Event detected*****\n\r");
65     printf(buffer);
66 #endif
67 }
68 #ifdef OUTPUT_ENC_DATA
69 static void Encrypt_data(uint8_t *input, size_t inSize, uint8_t *output, size_t *outSize);
70 static void Encrypt_data(uint8_t *input, size_t inSize, uint8_t *output, size_t *outSize)
71 {
72     psa_status_t psa_status;
73     size_t size = 0;
74
75     /* Encrypt single part functions */
76     psa_status = psa_cipher_encrypt(0x23, PSA_ALG_CBC_NO_PADDING, (uint8_t const*) input,
77                                     inSize,
78                                     output,
79                                     *outSize,
80                                     &size);
81     if (psa_status != PSA_SUCCESS)
82     {
83         printf("Error encrypting with the single-shot API\n\r");
84     }
85 }
```



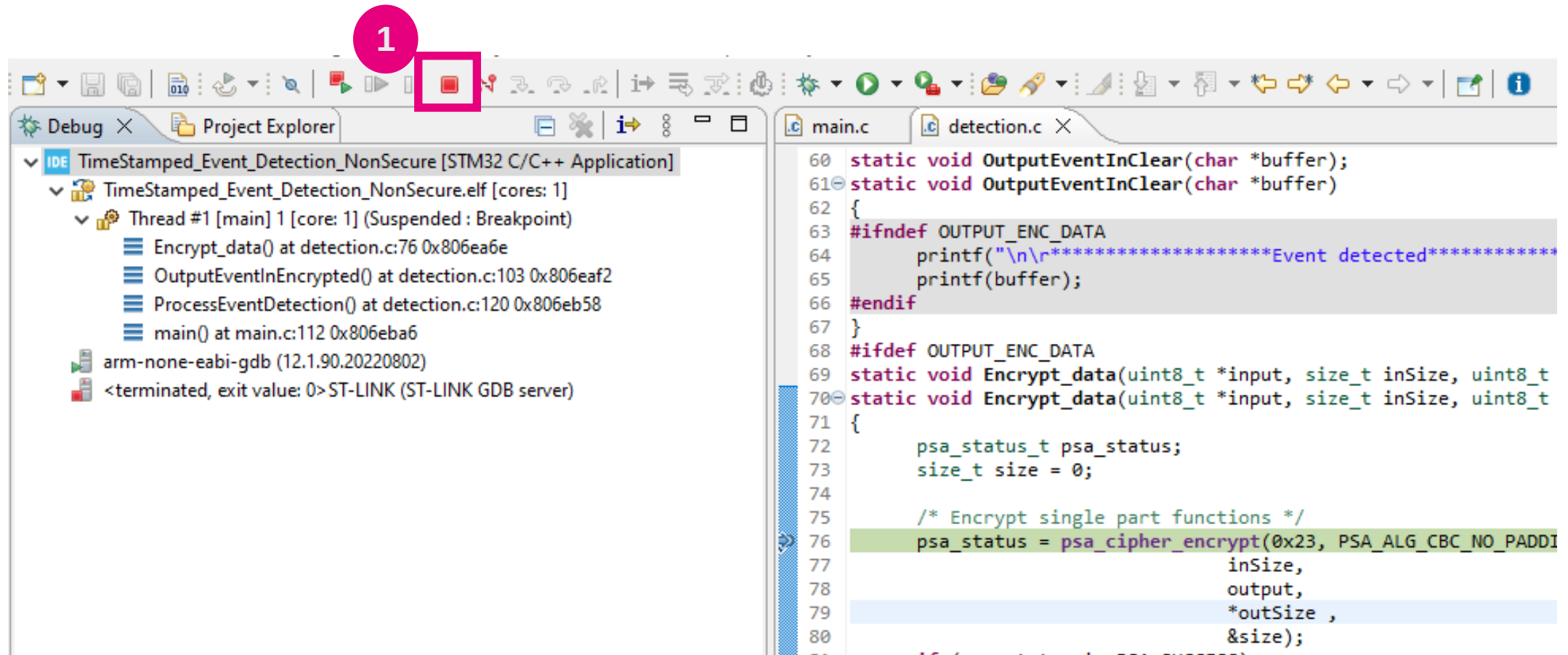
Implementation details

- PSA API used:

```
psa_status = psa_cipher_encrypt(0x23, // key_id
                                PSA_ALG_CBC_NO_PADDING, // cipher algorithm
                                (uint8_t const*) input, // Buffer containing the message to encrypt
                                inSize, //Size of the input buffer in bytes
                                output, // output buffer
                                *outSize, // Size of the output buffer in bytes
                                &size); //number of byte of the output
```

- A random IV has been generated for each encryption, this value has been added before the data in the output buffer.
- API reference
 - [10.4. Unauthenticated ciphers — PSA Crypto API 1.0.1 documentation \(arm-software.github.io\)](https://arm-software.github.io/PSA/1.0.1/docs/10.4.1/ciphers.html)
- STM32 MCU Wiki
 - [Security:SMAK for STM32H5 - stm32mcu](#)

Close the debug session





Conclusion

- We have seen how to provision a cryptographic key/data at production
- We have seen that thanks to CubeMX we manage to integrate a PSA API in our project.
- We know how to debug our implementation and used our first PSA API



What about our 12 securities function...



Our technology starts with You



Find out more at www.st.com

© STMicroelectronics - All rights reserved.

ST logo is a trademark or a registered trademark of STMicroelectronics International NV or its affiliates in the EU and/or other countries.

For additional information about ST trademarks, please refer to www.st.com/trademarks.

All other product or service names are the property of their respective owners.



life.augmented